

Universidad Pascual Bravo

Facultad de Ingeniería

Programa: Tecnología en Desarrollo de Software

Asignatura: Bases de datos I G(101)

Docente: Oralia Cortes Grajales

Tarea. Diseño de una base de datos con estándares y buenas prácticas

Estudiante: Juan Esteban Villada Ruiz

C.C 1039049872

Medellín – Colombia

27 septiembre 2026

INTRODUCCIÓN

A lo largo de la vida universitaria, he notado que la comunicación entre estudiantes suele estar muy dispersa o depender de canales informales como grupos de WhatsApp, donde la información importante rápidamente se pierde. Las plataformas oficiales de la institución son excelentes para la parte académica y administrativa, pero carecen de un espacio dedicado a la interacción social, la creación de comunidades de estudio espontáneas o el apoyo entre compañeros de diferentes semestres.

Pensando en esta necesidad, desarrollé el diseño lógico de la Red Social Pascualina. El objetivo de este proyecto no es solo crear una base de datos más, sino estructurar un sistema real y funcional donde los estudiantes puedan conectarse según sus intereses, compartir proyectos, organizar eventos de estudio y solicitar mentorías de manera sencilla.

En este documento presento el proceso completo mediante el cual llevé mi idea inicial (el modelo conceptual) hacia un modelo lógico totalmente normalizado hasta la Tercera Forma Normal (3FN). Explico en detalle la transformación de cada entidad a tablas en PostgreSQL, justifico el tipo de dato elegido para cada campo, describo cómo implementé las relaciones y presento las pruebas reales de funcionamiento que realicé en pgAdmin 4.

Análisis de Necesidades y Justificación del Caso

Para que la Red Social Pascualina responda a lo que realmente necesita la comunidad estudiantil, estructuré la base de datos para cubrir cinco áreas principales:

1. **Perfiles e Intereses:** Cada estudiante cuenta con su registro personal donde puede agregar su biografía, área de estudio y asociar sus habilidades técnicas y sus temas de interés.
2. **Red de Contactos y Mentorías:** Diseñe un sistema de seguimiento entre compañeros (seguidores y seguidos) y un espacio para solicitar mentorías entre estudiantes avanzados y principiantes.
3. **Publicaciones e Interacción:** Un muro principal para publicar dudas, noticias o recursos, con soporte para comentarios y reacciones (como "Me gusta" o "Interesante").

4. Grupos de Estudio: Espacios donde los estudiantes pueden unirse a clubes o grupos con intereses afines (programación, deportes, diseño) con roles asignados.
5. Eventos y Actividades: Creación de talleres o reuniones de estudio donde los miembros pueden confirmar su asistencia.

Referencia al Modelo Conceptual y Correspondencia de Entidades

Tomando como base el modelo conceptual que trabajé previamente, realicé la conversión a tablas relacionales. Para mantener buenas prácticas en SQL, renombré todas las entidades a minúsculas en singular y utilicé la convención **snake_case** (separando palabras con guion bajo) para evitar problemas de compatibilidad en PostgreSQL.

Tabla de Correspondencia entre Entidades y Tablas Lógicas

Entidad Conceptual	Tabla Lógica (SQL)	Justificación del Nombre y Conversión
Estudiante	estudiante	Conserva el nombre de la entidad principal. Guarda las credenciales y datos de acceso.
Perfil	perfil	Se conecta 1:1 con estudiante para almacenar la información detallada sin recargar la tabla principal.
Habilidad	habilidad	Funciona como un catálogo de destrezas (ej. Python, SQL, Liderazgo).
Interés	interes	Catálogo con áreas de gusto personal o académico.

Publicación	publicacion	Registra todo lo que los usuarios comparten en el muro.
Comentario	comentario	Almacena las respuestas o dudas sobre una publicación.
Reacción	reaccion	Guarda las interacciones de los usuarios en cada publicación.
Grupo	grupo	Maneja los clubes o comunidades creadas dentro de la red.
Evento	evento	Talleres o actividades programadas por los grupos o estudiantes.
Mentoría	mentoria	Gestiona el vínculo entre un mentor y un aprendiz.

Proceso de Normalización (1FN, 2FN y 3FN)

Para garantizar que mi base de datos no tenga datos duplicados ni errores al insertar, actualizar o eliminar información, apliqué las Formas Normales paso a paso:

- **Primera Forma Normal (1FN) - Datos atómicos:**
Me aseguré de que ningún campo contuviera listas de datos. Por ejemplo, en lugar de guardar en un solo campo del estudiante algo como "Python, Java, SQL", creé la tabla **habilidad** y conecté ambas entidades a través de una tabla intermedia. Además, separé el nombre completo en **nombre** y **apellido**.
- **Segunda Forma Normal (2FN) - Dependencia funcional completa:**
Garanticé que en las tablas asociativas con claves compuestas (como **estudiante_habilidad** o **miembro_grupo**), todos los datos adicionales dependan de la combinación de ambas claves. Por ejemplo, en **miembro_grupo**, el **rol** del usuario depende tanto de quién es el estudiante como de qué grupo se trata.

- **Tercera Forma Normal (3FN) - Sin dependencias transitivas:**
 Separé los campos que no dependían directamente de la clave primaria. Un ejemplo claro fue separar la tabla **perfil** de **estudiante**. De esta forma, la contraseña y el correo están en la tabla de inicio de sesión, mientras que la biografía y la carrera están en el perfil, evitando dependencias transitivas.
- .

Diccionario de Datos del Modelo Lógico

A continuación explico la estructura de las tablas más importantes de nuestro sistema, justificando por qué elegí cada tipo de dato:

Tabla Lógica: **estudiante**

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_estudiante	INT / SERIAL	Identificador único del estudiante.	Usé SERIAL para que PostgreSQL genere automáticamente los números enteros de forma autoincremental y eficiente.
nombre	VARCHAR(100)	Nombre(s) del estudiante.	VARCHAR(100) me da suficiente espacio para nombres compuestos sin desperdiciar almacenamiento.
apellido	VARCHAR(100)	Apellido(s) del estudiante.	VARCHAR(100) permite guardar dos apellidos completos cómodamente.

correo	VARCHAR(150)	Correo institucional.	Se definió como UNIQUE para que el sistema no permita registrar dos cuentas con el mismo correo.
password_hash	VARCHAR(255)	Contraseña encriptada.	Usé 255 caracteres para tener espacio suficiente al guardar la clave con algoritmos de encriptación seguros.

Tabla Lógica: perfil

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_perfil	INT / SERIAL	Identificador del perfil.	Clave primaria autoincremental para controlar cada registro.
id_estudiante	INT	Clave foránea del estudiante.	Es la clave foránea vinculada a estudiante . Le puse la condición UNIQUE para asegurar que cada estudiante tenga máximo un perfil.

area_estudio	VARCHAR(100)	Programa o facultad.	Texto flexible para guardar la carrera o facultad del estudiante.
biografia	TEXT	Descripción personal.	Elegí TEXT en lugar de VARCHAR para que el usuario pueda escribir una presentación libre sin un límite estricto de caracteres.
semestre	INT	Semestre actual.	Un número entero simple que permite ordenar o filtrar fácilmente por nivel académico.

Tabla Lógica: publicación

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_publicacion	INT / SERIAL	Identificador del post.	Clave primaria autoincremental.
id_estudiante	INT	ID del autor del post.	Clave foránea que relaciona la publicación directamente con el estudiante que la escribió.

contenido	TEXT	Texto de la publicación.	Usé TEXT para permitir desde mensajes cortos hasta publicaciones largas con código o explicaciones de proyectos.
fecha_publicacion	TIMESTAMP	Fecha y hora de creación.	Usé TIMESTAMP DEFAULT CURRENT_TIMESTAMP para que el sistema guarde la fecha y hora exactas en que se publicó sin tener que ponerla manualmente.

TABLAS CREADAS

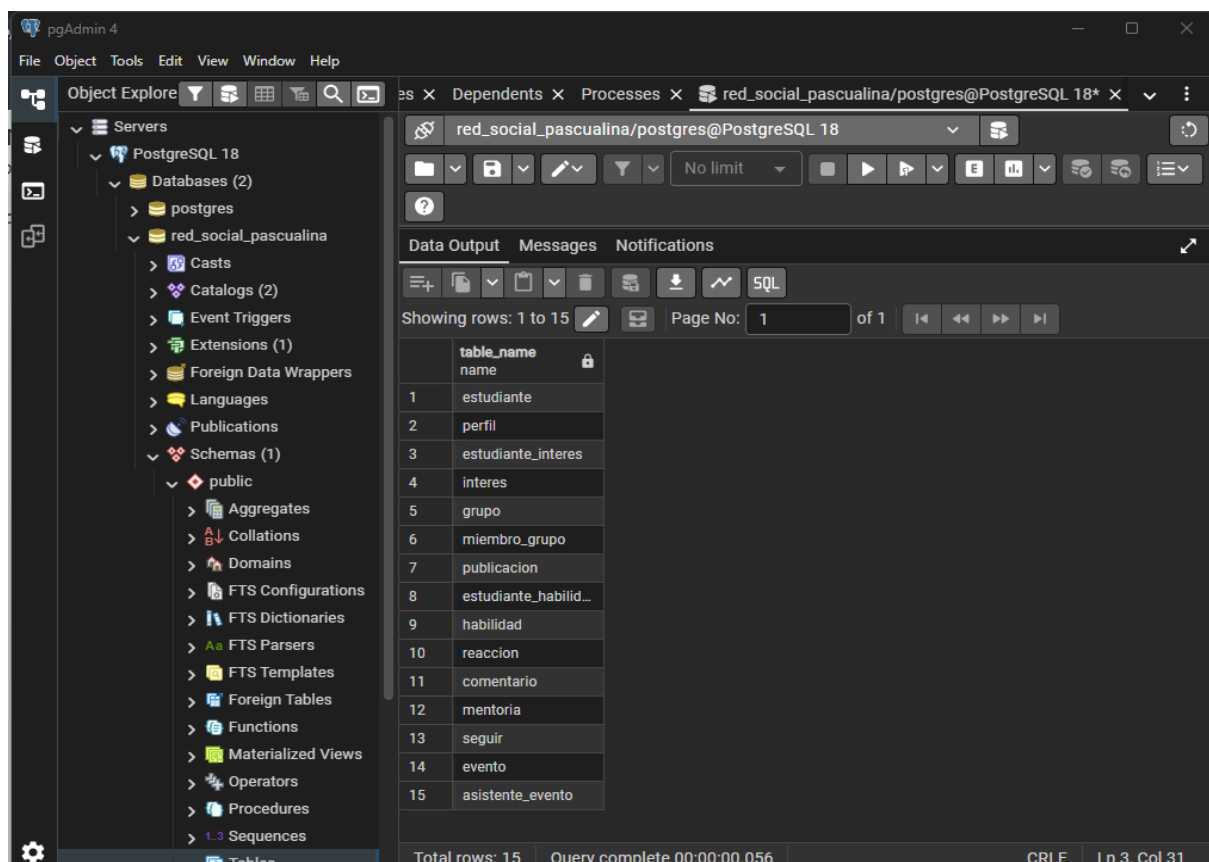


TABLA ESTUDIANTE.

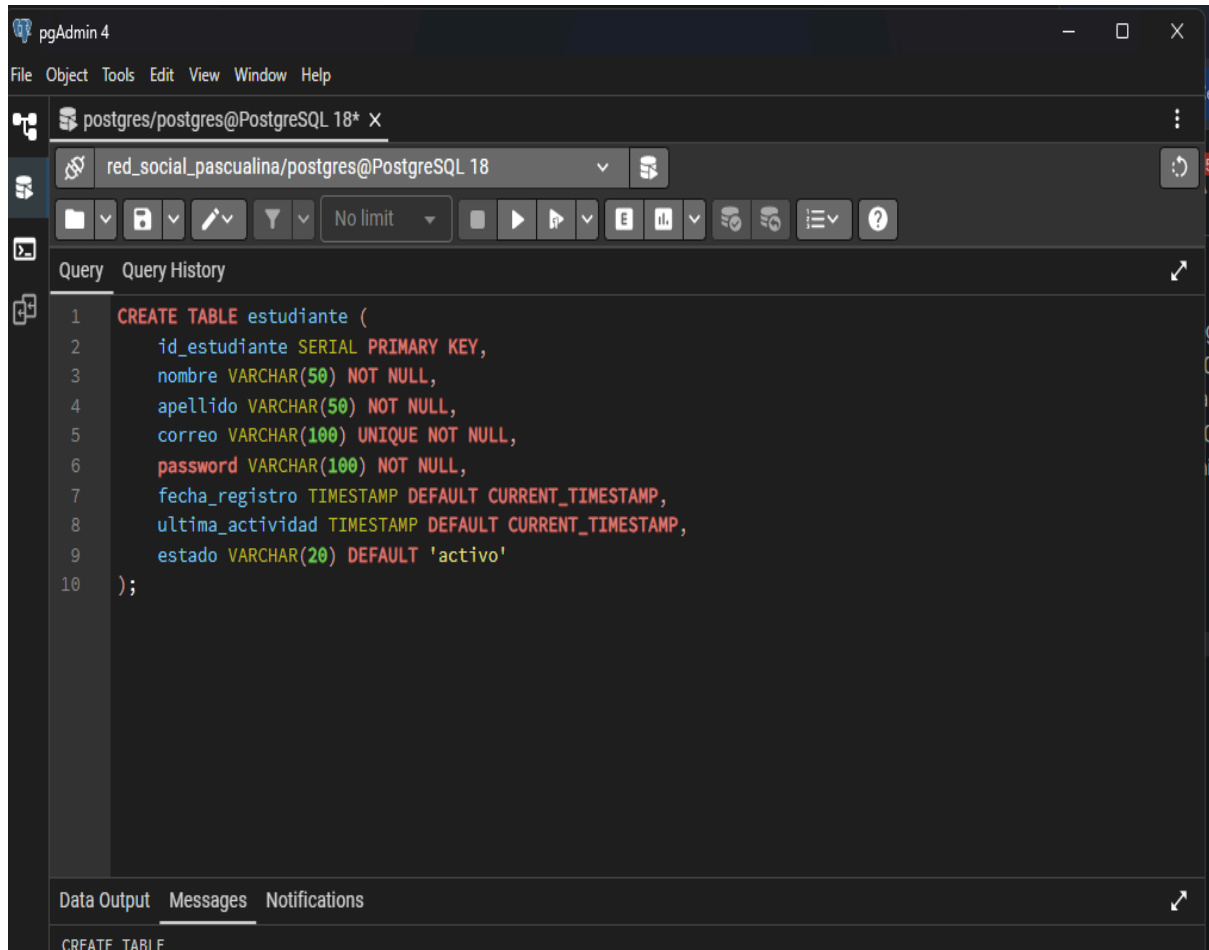


TABLA INTERESES

The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window displays the connection 'postgres/postgres@PostgreSQL 18*'. Below this, the 'Query' tab is active, showing a SQL query to create a table named 'interes'. The query is as follows:

```
1 CREATE TABLE interes (  
2     id_interes SERIAL PRIMARY KEY,  
3     nombre_interes VARCHAR(50) UNIQUE NOT NULL,  
4     categoria VARCHAR(30),  
5     descripcion TEXT  
6 )  
7 ;
```

The 'Data Output' tab is also visible, showing an error message: 'ERROR: la relación «interes» ya existe' and 'SQL state: 42P07'. The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.038' and 'CRLF Ln 2, Col 1'.

TABLA ESTUDIANTE_INTERESES

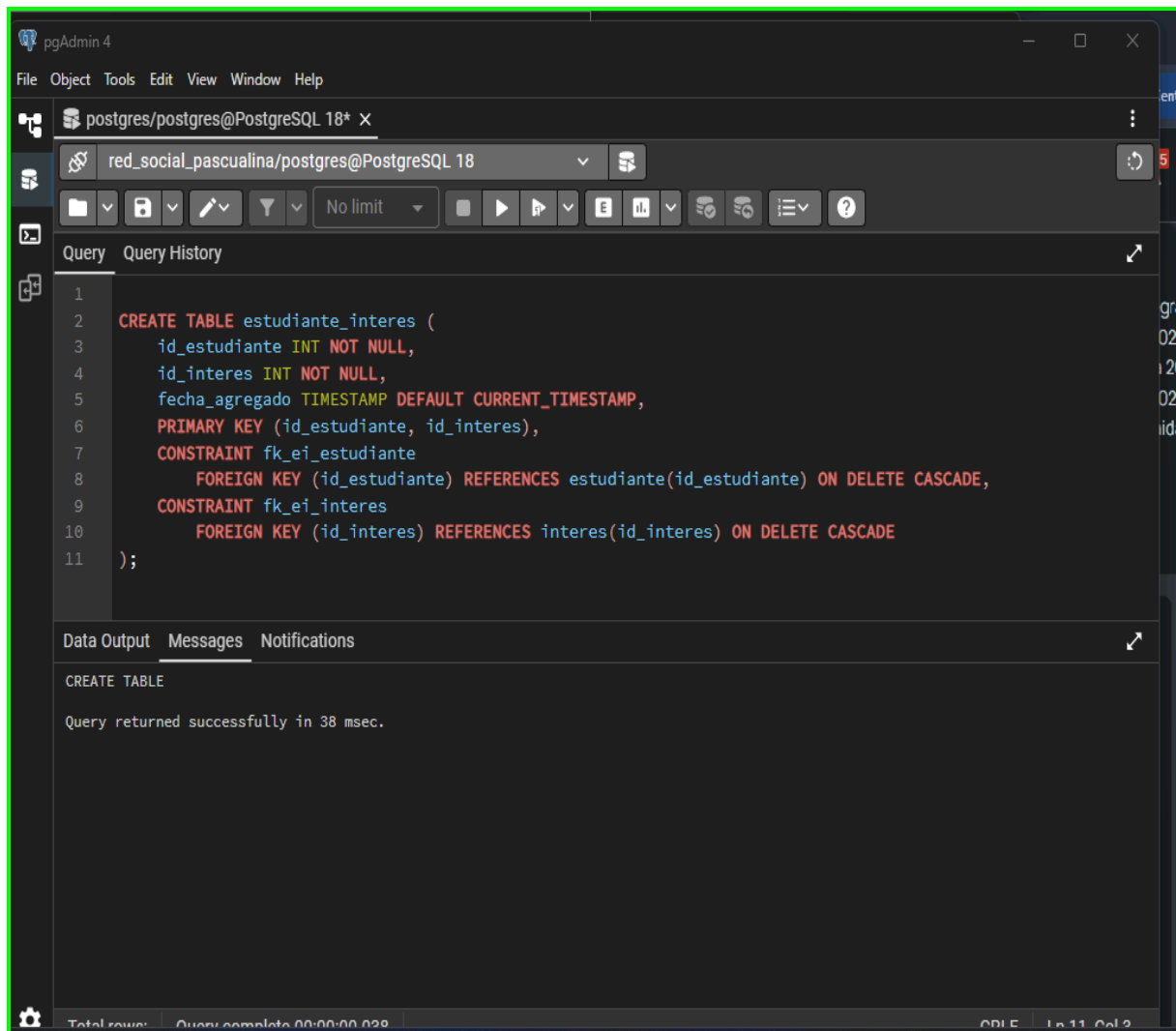


TABLA SEGUIR

The screenshot displays the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window shows a connection to 'postgres/postgres@PostgreSQL 18*'. The left sidebar contains icons for the Object Explorer, Query Tool, and Query History. The central pane shows the SQL editor with the following code:

```
1 CREATE TABLE seguir (  
2     id_seguidor INT NOT NULL,  
3     id_seguido INT NOT NULL,  
4     fecha_seguimiento TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
5     PRIMARY KEY (id_seguidor, id_seguido),  
6     CONSTRAINT fk_seguir_seguidor  
7         FOREIGN KEY (id_seguidor) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,  
8     CONSTRAINT fk_seguir_seguido  
9         FOREIGN KEY (id_seguido) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,  
10    CONSTRAINT chk_no_autoseguimiento  
11        CHECK (id_seguidor <> id_seguido)  
12 );  
13
```

Below the SQL editor, the 'Data Output' tab is active, showing the message: 'CREATE TABLE' and 'Query returned successfully in 34 msec.' The bottom status bar indicates 'Total rows: 0', 'Query complete 00:00:00.034', and 'Col 1'.

TABLA MENTORIA

The screenshot displays the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window shows a connection to 'postgres/postgres@PostgreSQL 18*'. The 'Query' tab is active, showing the following SQL code:

```
1 CREATE TABLE mentor (
2     id_mentor SERIAL PRIMARY KEY,
3     id_mentor INT NOT NULL,
4     id_estudiante INT NOT NULL,
5     area_conocimiento VARCHAR(50) NOT NULL,
6     estado VARCHAR(20) DEFAULT 'pendiente',
7     fecha_solicitud TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
8     fecha_inicio TIMESTAMP,
9     fecha_fin TIMESTAMP,
10    CONSTRAINT fk_mentor_mentor
11        FOREIGN KEY (id_mentor) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,
12    CONSTRAINT fk_mentor_estudiante
13        FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,
14    CONSTRAINT chk_no_automentoria
15        CHECK (id_mentor <> id_estudiante)
16 );
17
```

The 'Data Output' tab is also visible, showing the message: 'CREATE TABLE' and 'Query returned successfully in 40 msec.' A green notification box at the bottom right confirms: 'Query returned successfully in 40 msec.'

TABLA PUBLICACION

The screenshot shows the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window displays the connection 'postgres/postgres@PostgreSQL 18*' and the server 'red_social_pascualina/postgres@PostgreSQL 18'. The 'Query' tab is active, showing the following SQL code:

```
1 CREATE TABLE publicacion (  
2     id_publicacion SERIAL PRIMARY KEY,  
3     id_estudiante INT NOT NULL,  
4     contenido TEXT NOT NULL,  
5     tipo_publicacion VARCHAR(50) DEFAULT 'general',  
6     url_adjunto VARCHAR(255),  
7     fecha_publicacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
8     visibilidad VARCHAR(20) DEFAULT 'publico',  
9     CONSTRAINT fk_publicacion_estudiante  
10     FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE  
11 );
```

The 'Data Output' tab is also visible, showing the message: 'CREATE TABLE' and 'Query returned successfully in 38 msec.' The bottom status bar indicates 'Total rows: Query complete 00:00:00.028' and 'Col 1'.

COMENTARIO

The screenshot displays the pgAdmin 4 web interface. The top navigation bar includes 'File', 'Object', 'Tools', 'Edit', 'View', 'Window', and 'Help'. The main toolbar contains icons for file operations, query execution, and other database management tasks. The 'Query' tab is active, showing a SQL script to create a table named 'comentario'. The script includes columns for 'id_comentario' (SERIAL PRIMARY KEY), 'id_publicacion' (INT NOT NULL), 'id_estudiante' (INT NOT NULL), 'contenido' (TEXT NOT NULL), and 'fecha_comentario' (TIMESTAMP DEFAULT CURRENT_TIMESTAMP). It also defines two foreign key constraints: 'fk_comentario_publicacion' and 'fk_comentario_estudiante', both with ON DELETE CASCADE. The 'Data Output' tab is selected, showing the message 'Query returned successfully in 35 msec.'

```
1 CREATE TABLE comentario (  
2     id_comentario SERIAL PRIMARY KEY,  
3     id_publicacion INT NOT NULL,  
4     id_estudiante INT NOT NULL,  
5     contenido TEXT NOT NULL,  
6     fecha_comentario TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7     CONSTRAINT fk_comentario_publicacion  
8         FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion) ON DELETE CASCADE,  
9     CONSTRAINT fk_comentario_estudiante  
10        FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE  
11 );
```

Query returned successfully in 35 msec.

REACCION

The screenshot shows a PostgreSQL IDE interface with a dark theme. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The title bar indicates the connection is to 'postgres/postgres@PostgreSQL 18*'. Below the title bar, the database 'red_social_pascualina/postgres@PostgreSQL 18' is selected. A toolbar with various icons for file operations and query execution is visible. The main editor area is divided into 'Query' and 'Query History' tabs. The 'Query' tab contains a SQL script to create a table named 'reaccion'. The script includes columns for 'id_reaccion' (SERIAL PRIMARY KEY), 'id_publicacion' (INT NOT NULL), 'id_estudiante' (INT NOT NULL), 'tipo_reaccion' (VARCHAR(30) NOT NULL), and 'fecha_reaccion' (TIMESTAMP DEFAULT CURRENT_TIMESTAMP). It also includes three constraints: a unique constraint 'uq_estudiante_publicacion' on 'id_estudiante' and 'id_publicacion', and two foreign key constraints 'fk_reaccion_publicacion' and 'fk_reaccion_estudiante' referencing 'publicacion' and 'estudiante' respectively. The 'Messages' tab at the bottom shows the successful execution of the 'CREATE TABLE' query, returning the message 'Query returned successfully in 35 msec.' The status bar at the bottom indicates 'Total rows: Query complete 00:00:00.035'.

```
1 CREATE TABLE reaccion (  
2     id_reaccion SERIAL PRIMARY KEY,  
3     id_publicacion INT NOT NULL,  
4     id_estudiante INT NOT NULL,  
5     tipo_reaccion VARCHAR(30) NOT NULL,  
6     fecha_reaccion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7     CONSTRAINT uq_estudiante_publicacion UNIQUE (id_estudiante, id_publicacion),  
8     CONSTRAINT fk_reaccion_publicacion  
9         FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion) ON DELETE CASCADE,  
10    CONSTRAINT fk_reaccion_estudiante  
11        FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE  
12 );
```

CREATE TABLE

Query returned successfully in 35 msec.

Total rows: Query complete 00:00:00.035

GRUPO

The screenshot displays the pgAdmin 4 application window. The title bar reads "pgAdmin 4". The menu bar includes "File", "Object", "Tools", "Edit", "View", "Window", and "Help". The main toolbar shows icons for file operations, query execution, and other database management tasks. The "Query" tab is active, showing a SQL script to create a table named "grupo". The script includes columns for "id_grupo" (SERIAL PRIMARY KEY), "nombre_grupo" (VARCHAR(100) NOT NULL), "descripcion" (TEXT), "id_creador" (INT NOT NULL), and "fecha_creacion" (TIMESTAMP DEFAULT CURRENT_TIMESTAMP). It also features a foreign key constraint "fk_grupo_creador" that references the "id_estudiante" column in the "estudiante" table with ON DELETE RESTRICT. The "Data Output" tab is selected, showing the message "CREATE TABLE" and "Query returned successfully in 36 msec." The status bar at the bottom indicates "Total rows: 0" and "Query complete 00:00:00.036".

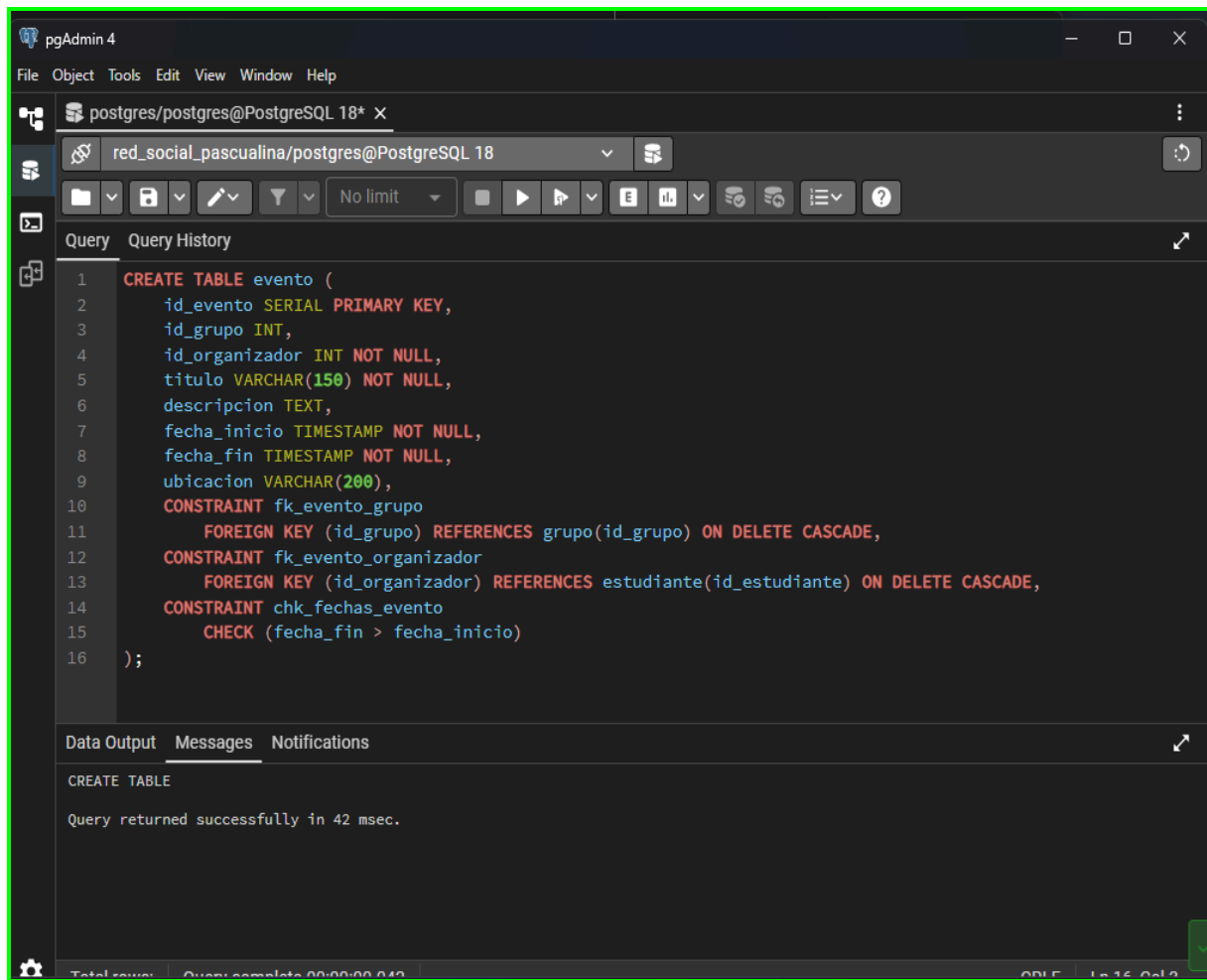
```
1 CREATE TABLE grupo (  
2     id_grupo SERIAL PRIMARY KEY,  
3     nombre_grupo VARCHAR(100) NOT NULL,  
4     descripcion TEXT,  
5     id_creador INT NOT NULL,  
6     fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7     CONSTRAINT fk_grupo_creador  
8         FOREIGN KEY (id_creador) REFERENCES estudiante(id_estudiante) ON DELETE RESTRICT  
9 );
```

CREATE TABLE

Query returned successfully in 36 msec.

Total rows: 0 Query complete 00:00:00.036

EVENTO



The screenshot displays the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window shows a connection to 'postgres/postgres@PostgreSQL 18*'. The 'Query' tab is active, displaying a SQL script to create a table named 'evento'. The script includes columns for 'id_evento' (SERIAL PRIMARY KEY), 'id_grupo' (INT), 'id_organizador' (INT NOT NULL), 'titulo' (VARCHAR(150) NOT NULL), 'descripcion' (TEXT), 'fecha_inicio' (TIMESTAMP NOT NULL), 'fecha_fin' (TIMESTAMP NOT NULL), and 'ubicacion' (VARCHAR(200)). It also features three constraints: a foreign key 'fk_evento_grupo' referencing 'grupo(id_grupo)', a foreign key 'fk_evento_organizador' referencing 'estudiante(id_estudiante)', and a check constraint 'chk_fechas_evento' ensuring 'fecha_fin > fecha_inicio'. The 'Messages' tab at the bottom shows the successful execution of the 'CREATE TABLE' query, stating 'Query returned successfully in 42 msec.'.

```
1 CREATE TABLE evento (  
2     id_evento SERIAL PRIMARY KEY,  
3     id_grupo INT,  
4     id_organizador INT NOT NULL,  
5     titulo VARCHAR(150) NOT NULL,  
6     descripcion TEXT,  
7     fecha_inicio TIMESTAMP NOT NULL,  
8     fecha_fin TIMESTAMP NOT NULL,  
9     ubicacion VARCHAR(200),  
10    CONSTRAINT fk_evento_grupo  
11        FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE,  
12    CONSTRAINT fk_evento_organizador  
13        FOREIGN KEY (id_organizador) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE,  
14    CONSTRAINT chk_fechas_evento  
15        CHECK (fecha_fin > fecha_inicio)  
16 );
```

CREATE TABLE
Query returned successfully in 42 msec.

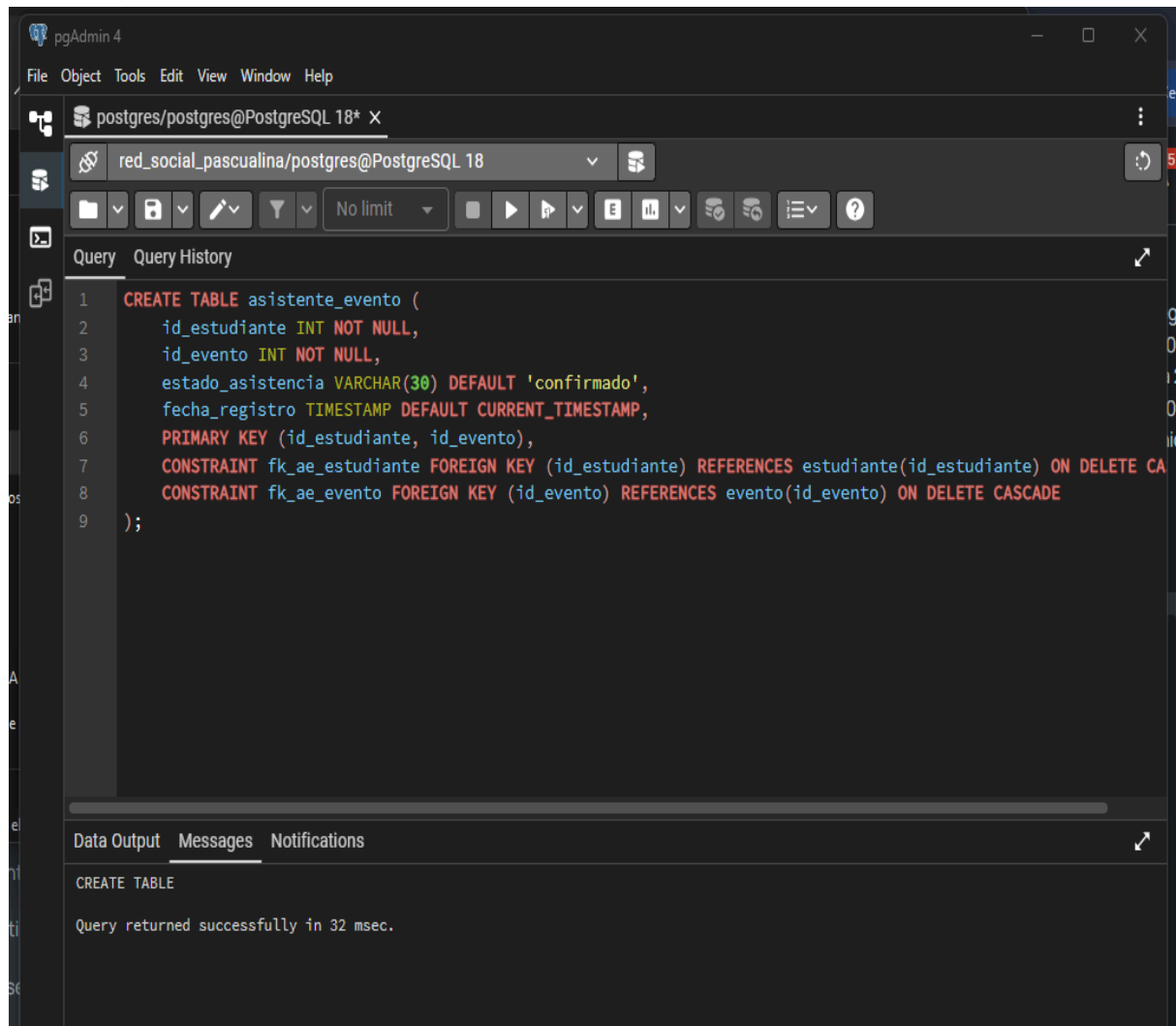
MIEMBRO_GRUPO

The screenshot displays the pgAdmin 4 interface. The top menu bar includes File, Object, Tools, Edit, View, Window, and Help. The main window shows a connection to 'postgres/postgres@PostgreSQL 18*'. Below this, a toolbar contains icons for file operations, query execution, and other database functions. The 'Query' tab is active, showing the following SQL script:

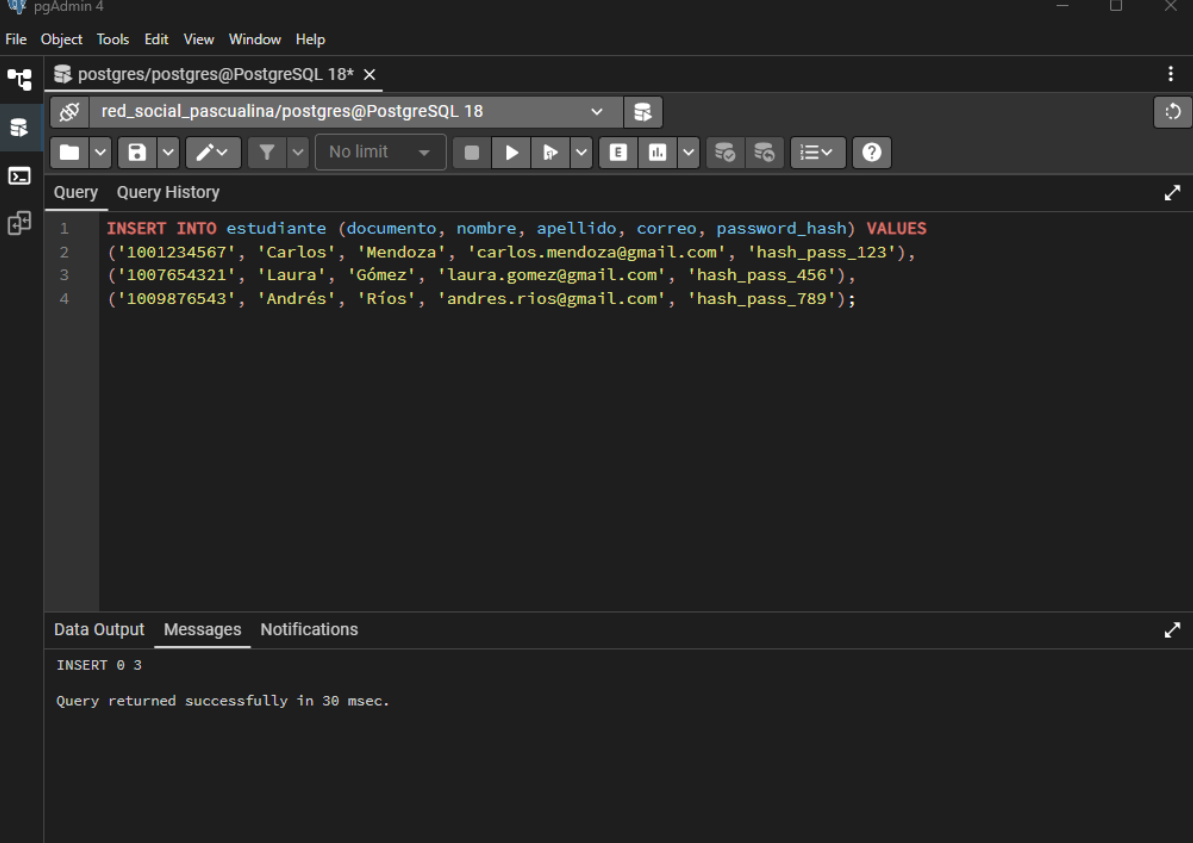
```
1 CREATE TABLE miembro_grupo (  
2     id_estudiante INT NOT NULL,  
3     id_grupo INT NOT NULL,  
4     fecha_union TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
5     rol VARCHAR(30) DEFAULT 'miembro',  
6     PRIMARY KEY (id_estudiante, id_grupo),  
7     CONSTRAINT fk_mg_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CA  
8     CONSTRAINT fk_mg_grupo FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo) ON DELETE CASCADE  
9 );
```

Below the query editor, the 'Data Output' tab is selected, displaying the message: 'CREATE TABLE' and 'Query returned successfully in 51 msec.'

ASISTENTE_EVENTO



INSERT INTO



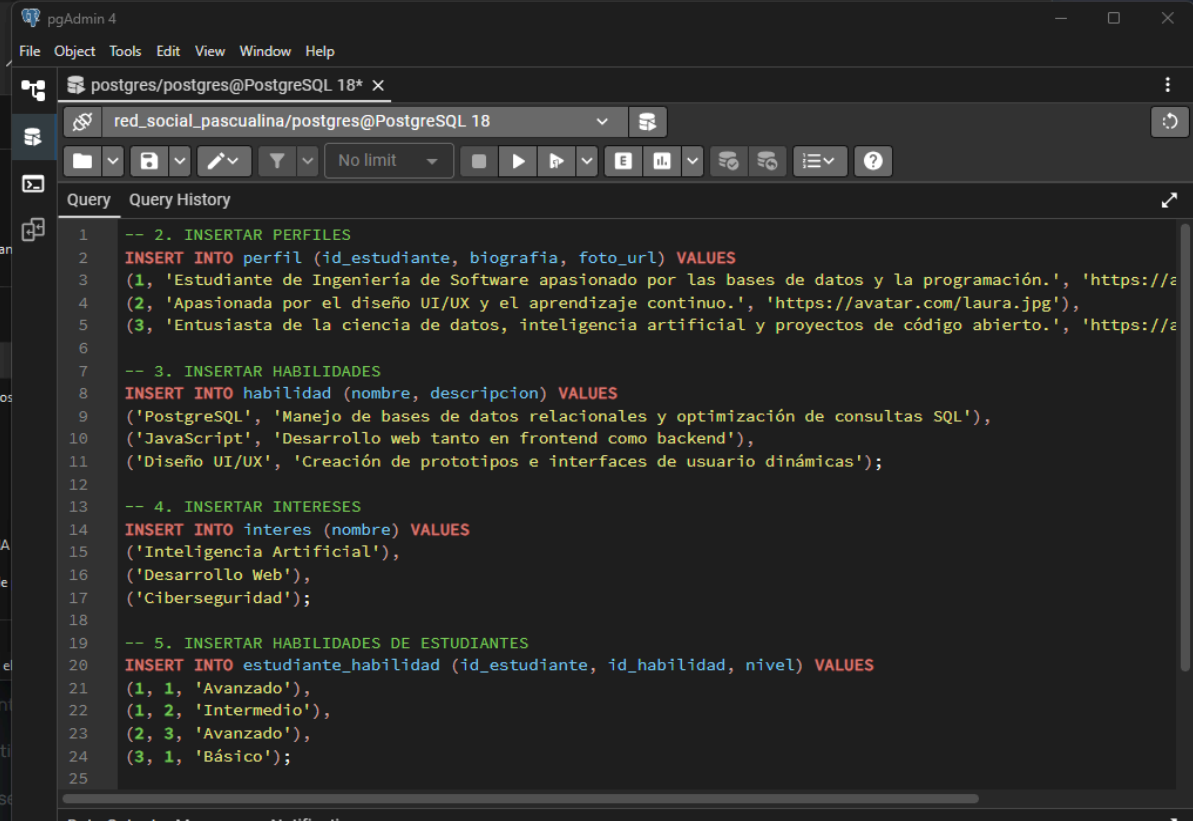
The screenshot shows the pgAdmin 4 interface. The top toolbar includes icons for file operations, database connections, and query execution. The main query editor displays an SQL INSERT statement. The 'Messages' tab at the bottom shows the execution result.

```
1 INSERT INTO estudiante (documento, nombre, apellido, correo, password_hash) VALUES
2 ('1001234567', 'Carlos', 'Mendoza', 'carlos.mendoza@gmail.com', 'hash_pass_123'),
3 ('1007654321', 'Laura', 'Gómez', 'laura.gomez@gmail.com', 'hash_pass_456'),
4 ('1009876543', 'Andrés', 'Ríos', 'andres.rios@gmail.com', 'hash_pass_789');
```

Messages

INSERT 0 3

Query returned successfully in 30 msec.



The screenshot shows the pgAdmin 4 interface with a multi-line SQL query. The 'Messages' tab at the bottom is empty.

```
1 -- 2. INSERTAR PERFILES
2 INSERT INTO perfil (id_estudiante, biografia, foto_url) VALUES
3 (1, 'Estudiante de Ingeniería de Software apasionado por las bases de datos y la programación.', 'https://a
4 (2, 'Apasionada por el diseño UI/UX y el aprendizaje continuo.', 'https://avatar.com/laura.jpg'),
5 (3, 'Entusiasta de la ciencia de datos, inteligencia artificial y proyectos de código abierto.', 'https://a
6
7 -- 3. INSERTAR HABILIDADES
8 INSERT INTO habilidad (nombre, descripcion) VALUES
9 ('PostgreSQL', 'Manejo de bases de datos relacionales y optimización de consultas SQL'),
10 ('JavaScript', 'Desarrollo web tanto en frontend como backend'),
11 ('Diseño UI/UX', 'Creación de prototipos e interfaces de usuario dinámicas');
12
13 -- 4. INSERTAR INTERESES
14 INSERT INTO interes (nombre) VALUES
15 ('Inteligencia Artificial'),
16 ('Desarrollo Web'),
17 ('Ciberseguridad');
18
19 -- 5. INSERTAR HABILIDADES DE ESTUDIANTES
20 INSERT INTO estudiante_habilidad (id_estudiante, id_habilidad, nivel) VALUES
21 (1, 1, 'Avanzado'),
22 (1, 2, 'Intermedio'),
23 (2, 3, 'Avanzado'),
24 (3, 1, 'Básico');
25
```

Messages

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

red_social_pascualina/postgres@PostgreSQL 18

No limit

Query Query History

```
1 -- 1. Insertar tipos de intereses
2 INSERT INTO interes (nombre, descripcion) VALUES
3 ('Programación Web', 'Desarrollo de aplicaciones y sitios web'),
4 ('Inteligencia Artificial', 'Modelos de Machine Learning y algoritmos'),
5 ('Diseño UI/UX', 'Diseño de interfaces y experiencia de usuario'),
6 ('Bases de Datos', 'Modelado y administración de datos SQL');
7
8 -- 2. Asignar los intereses a los estudiantes
9 INSERT INTO estudiante_interes (id_estudiante, id_interes) VALUES
10 (5, 1), -- Carlos le interesa Programación Web
11 (5, 4), -- Carlos le interesa Bases de Datos
12 (6, 3), -- Laura le interesa Diseño UI/UX
13 (7, 2), -- Andrés le interesa Inteligencia Artificial
14 (7, 4); -- Andrés le interesa Bases de Datos
```

Data Output Messages Notifications

INSERT 0 3

Query returned successfully in 45 msec.

Total rows: Query complete 00:00:00.045 0/15 Ln 14 Col 45

pgAdmin 4

File Object Tools Edit View Window Help

postgres/postgres@PostgreSQL 18* X

red_social_pascualina/postgres@PostgreSQL 18

No limit

Query Query History

```
1 INSERT INTO publicacion (id_estudiante, contenido) VALUES
2 (5, '¡Hola a todos! Bienvenidos a la red social Pascualina.'),
3 (6, '¿Alguien interesado en formar un grupo de estudio de Bases de Datos?'),
4 (7, 'Comparto un recurso excelente para aprender PostgreSQL y PL/pgSQL.');
```

Data Output Messages Notifications

INSERT 0 3

Query returned successfully in 46 msec.

Descripción de las Relaciones del Modelo

Relación 1:1 — Estudiante posee Perfil

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante - Perfil	estudiante perfil	1:1	Agregué <code>id_estudiante</code> en la tabla <code>perfil</code> con la restricción <code>UNIQUE</code> y <code>FOREIGN KEY</code> .	Asegura que un perfil pertenezca a un solo estudiante y que ningún usuario pueda crear múltiples perfiles.

Relación 1:N — Estudiante crea Publicaciones

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Usuario Publica Post	estudiante publicacion	1:N	Coloqué <code>id_estudiante</code> como clave foránea dentro de la tabla <code>publicacion</code> .	Permite que un estudiante escriba todas las publicaciones que quiera, pero cada publicación pertenece a un solo autor.

Relación N:M — Estudiante pertenece a Grupos

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Miembros de Grupo	estudiante grupo	N:M	Creé la tabla intermedia <code>miembro_grupo</code> uniendo <code>id_estudiante</code> e <code>id_grupo</code> como clave primaria compuesta.	Rompe la relación muchos a muchos y da el espacio perfecto para guardar datos adicionales, como la fecha de ingreso y el rol del usuario en el grupo.

Relación N:M — Red de Seguidores (Autorreferencial)

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación

Seguimiento Social	estudiante seguir	N:M	Creé la tabla seguir con los campos id_seguidor e id_seguido , ambos apuntando a estudiante .	Permite construir la red de seguidores entre los mismos usuarios. Le agregué una regla para evitar que un estudiante se siga a sí mismo (id_seguidor <> id_seguido).
---------------------------	--------------------------	------------	---	---

Conclusión

Durante el desarrollo de esta tarea, el aspecto más enriquecedor fue aplicar el proceso de normalización paso a paso desde el modelo conceptual. Al inicio, la mayor dificultad estuvo en definir correctamente las tablas intermedias para las relaciones muchos a muchos (N:M) y configurar adecuadamente las restricciones de integridad referencial (**ON DELETE CASCADE**). Comprobar en pgAdmin 4 que la base de datos mantiene la consistencia al realizar consultas compuestas con **JOIN** y que previene la inserción de registros duplicados demostró la importancia de una buena estructuración lógica antes del despliegue en un entorno real.

Enlace al Video de Sustentación

LINK VIDEO: [VIDEO DISEÑO BASE DE DATOS](#)

LINK GitHub:

<https://github.com/juanvillada872-debug/bd1-redsocial-grupo2/new/main/Tarea2/informe/Video>